

R4DS 06 – Exploratory Data Analysis

Introduction

What is EDA?

“Far better an approximate answer to the right question, which is often vague, than an exact answer to the wrong question.” — John Tukey

EDA is an **iterative cycle**:

- 1 Generate **questions** about your data
- 2 **Visualize**, transform, and model to search for answers
- 3 **Refine** your questions based on what you learn

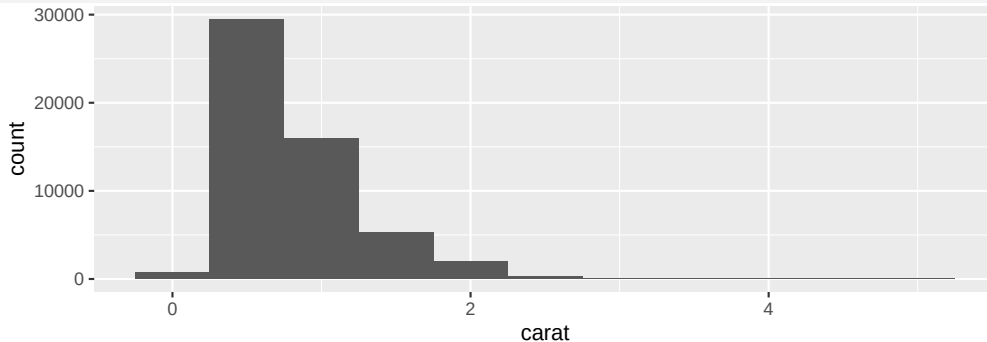
Two universal questions:

- What type of **variation** occurs within my variables?
- What type of **covariation** occurs between my variables?

Variation

Visualizing Distributions

```
ggplot(diamonds, aes(x = carat)) +  
  geom_histogram(binwidth = 0.5)
```

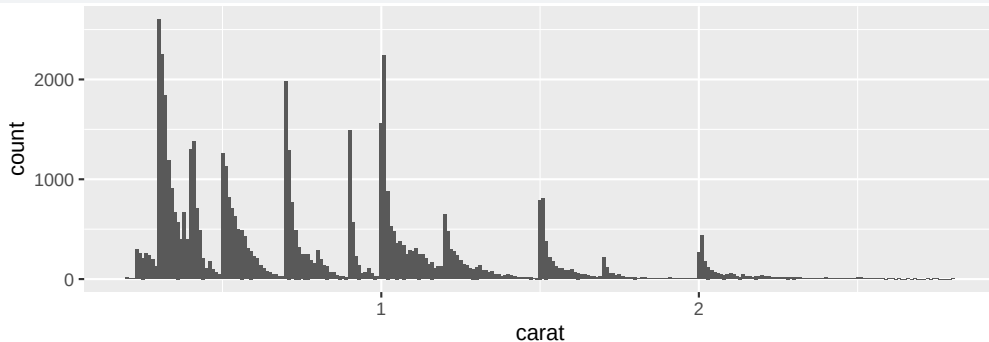


Right-skewed: most diamonds are small. Peaks at round values (0.5, 1, 1.5, 2).

Zoom In: Smaller Binwidth

```
smaller <- diamonds |> filter(carat < 3)

ggplot(smaller, aes(x = carat)) +
  geom_histogram(binwidth = 0.01)
```



Spikes at whole/half carats — rounding by sellers. More diamonds just above each round number.

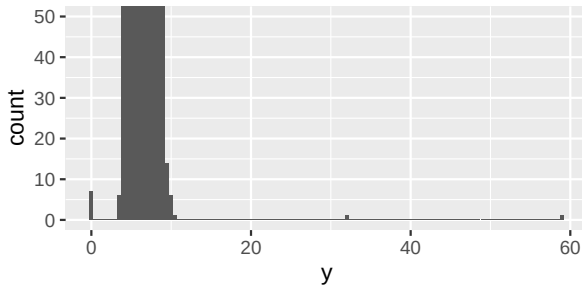
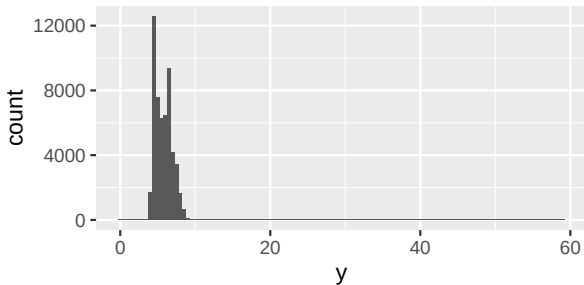
When exploring a distribution, look for:

- **Typical values:** Which are most common? Why?
- **Rare values:** Which are rare? Does that match expectations?
- **Unusual patterns:** Clusters, spikes, gaps?
- **Subgroups:** Do clusters suggest distinct populations?

Follow-up: How are observations within clusters similar? How do clusters differ?

Unusual Values (Outliers)

```
ggplot(diamonds, aes(x = y)) +  
  geom_histogram(binwidth = 0.5)  
ggplot(diamonds, aes(x = y)) +  
  geom_histogram(binwidth = 0.5) +  
  coord_cartesian(ylim = c(0, 50))
```



`coord_cartesian()` zooms without dropping data. Reveals outliers at 0, ~30, ~60 mm.

Identifying Outliers

```
diamonds |>
  filter(y < 3 | y > 20) |>
  select(price, x, y, z) |>
  arrange(y)
```

```
#> # A tibble: 9 x 4
#>   price      x      y      z
#>   <int> <dbl> <dbl> <dbl>
#> 1  5139  0      0      0
#> 2  6381  0      0      0
#> 3 12800  0      0      0
#> 4 15686  0      0      0
#> 5 18034  0      0      0
#> ... [output truncated]
```

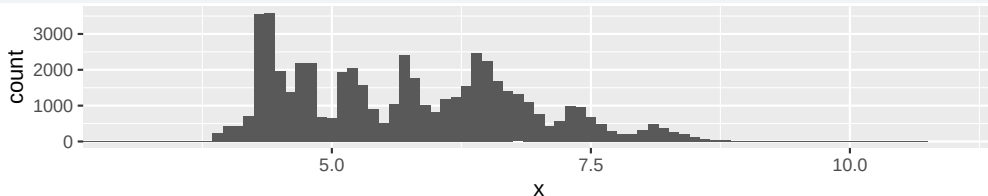
Width of 0 mm = data entry error. 32 mm and 59 mm = implausible for these prices.

Exercises (Variation)

- 1 Explore the distribution of `x`, `y`, and `z` in `diamonds`. What do you learn?
- 2 Explore the distribution of `price`. Anything unusual? (Hint: try many binwidths.)
- 3 How many diamonds are 0.99 carat? How many are 1 carat? Why the difference?
- 4 Compare `coord_cartesian()` vs. `xlim()/ylim()` when zooming on a histogram.

1

```
ggplot(diamonds, aes(x = x)) +  
  geom_histogram(binwidth = 0.1) +  
  coord_cartesian(xlim = c(3, 11))
```



x and **y** are similar (length/width); **z** is smaller (depth). All have outliers near 0 and at high extremes.

② No diamonds priced around \$1,500 — a gap in the distribution. Visible with small binwidth.

③

```
diamonds |> filter(carat == 0.99) |> nrow()
```

```
#> [1] 23
```

```
diamonds |> filter(carat == 1) |> nrow()
```

```
#> [1] 1558
```

23 vs. 1,558 — sellers round up to 1 carat (higher perceived value).

④ `coord_cartesian()` zooms visually; `xlim()/ylim()` **drops data** outside the range (changes bin counts).

Handling Unusual Values

Replace, Don't Drop

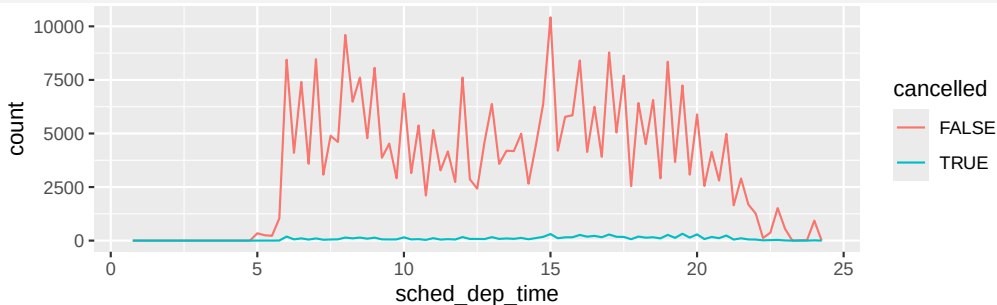
```
# BAD: drops entire row
diamonds2 <- diamonds |> filter(between(y, 3, 20))

# GOOD: replace outliers with NA
diamonds2 <- diamonds |>
  mutate(y = if_else(y < 3 | y > 20, NA, y))
```

Don't drop rows — one bad value doesn't invalidate the whole observation. Use `if_else()` to recode outliers as `NA`.

Working with Missing Values

```
nycflights13::flights |>
  mutate(
    cancelled = is.na(dep_time),
    sched_hour = sched_dep_time %/% 100,
    sched_min = sched_dep_time %% 100,
    sched_dep_time = sched_hour + sched_min / 60
  ) |>
  ggplot(aes(x = sched_dep_time)) +
  geom_freqpoly(aes(color = cancelled), binwidth = 1/4)
```



Exercises (Missing Values)

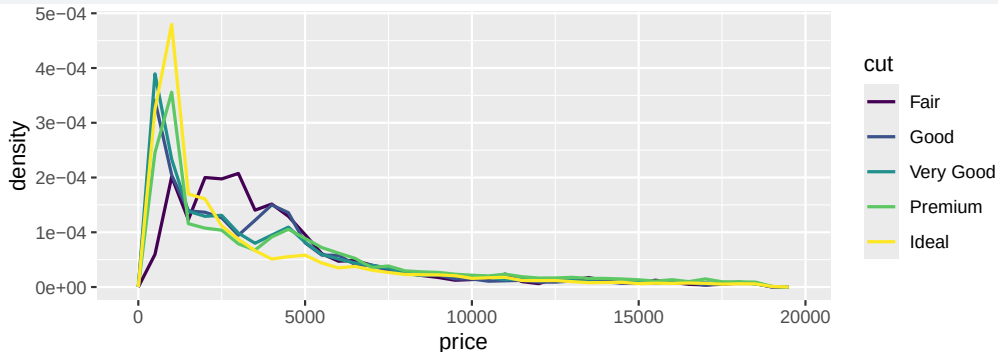
- 1 What happens to missing values in a histogram? In a bar chart? Why the difference?
- 2 What does `na.rm = TRUE` do in `mean()` and `sum()`?

- ① Histograms **silently drop** NAs (with a warning). Bar charts show NAs as a separate category. Histograms need numeric bins; NAs have no position. Bar charts treat NAs as another level.
- ② `na.rm = TRUE` removes NAs before computing. Without it, `mean()` and `sum()` return `NA` if any input is missing.

Covariation

Categorical + Numerical: Frequency Polygon

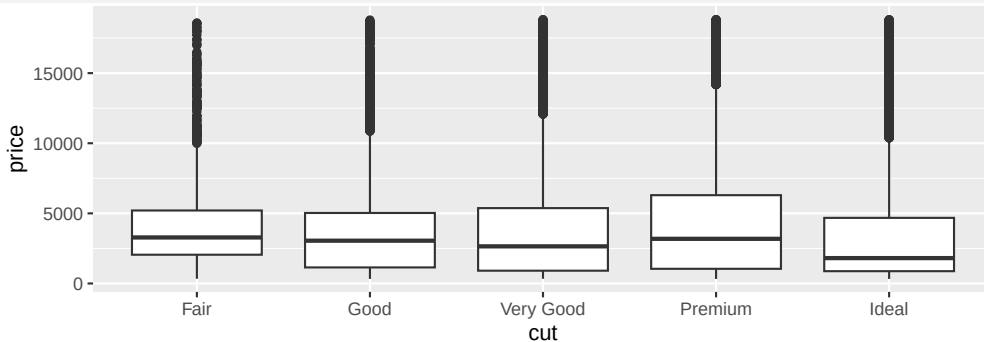
```
ggplot(diamonds, aes(x = price, y = after_stat(density))) +  
  geom_freqpoly(aes(color = cut), binwidth = 500,  
    linewidth = 0.75)
```



`after_stat(density)` standardizes area to 1. Surprise: Fair diamonds appear most expensive!

Categorical + Numerical: Boxplot

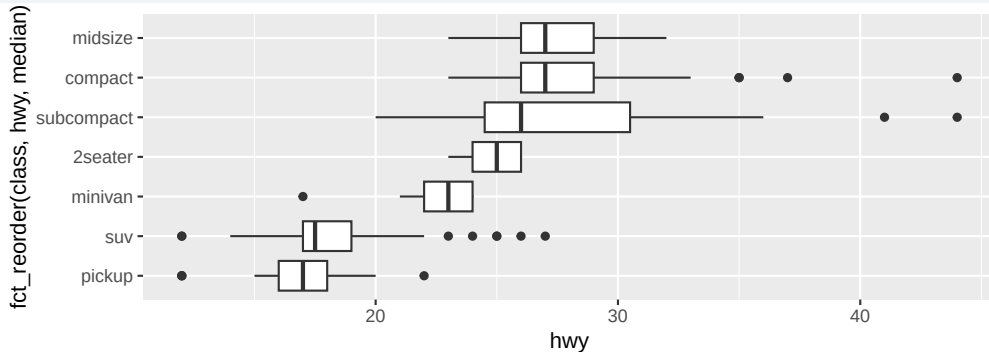
```
ggplot(diamonds, aes(x = cut, y = price)) +  
  geom_boxplot()
```



Confirms: lower quality → higher median price. (Confounded by carat — explained later.)

Reordering with `fct_reorder()`

```
ggplot(mpg, aes(x = hwy,  
  y = fct_reorder(class, hwy, median))) +  
  geom_boxplot()
```



`fct_reorder(class, hwy, median)` sorts categories by median `hwy`. Swap axes for long labels.

Exercises (Categorical + Numerical)

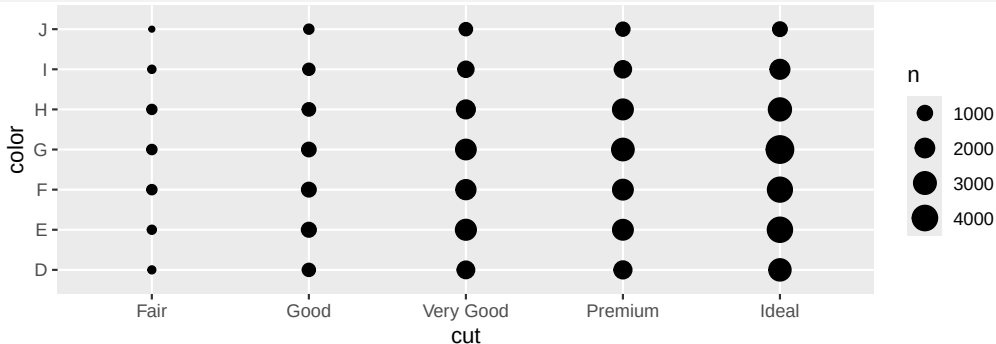
- 1 Improve the cancelled vs. non-cancelled flights visualization.
- 2 Which variable best predicts diamond price? How is it correlated with `cut`? Why does this make lower-quality diamonds more expensive?
- 3 Try `coord_flip()` vs. swapping `x/y`.
- 4 Compare: `geom_violin()`, faceted `geom_histogram()`, colored `geom_freqpoly()`, colored `geom_density()`.

- ① Use density or facet with `scales = "free_y"` to handle the count imbalance.
- ② **Carat** predicts price best. Larger (lower-cut) diamonds tend to weigh more. When controlling for carat, better cut → higher price. Simpson's paradox: the confounding variable (carat) reverses the apparent relationship.

- ③ Same result visually. Swapping `x/y` is more explicit; `coord_flip()` is convenient for modifying existing code.
- ④ `geom_violin()`: shows full shape, compact. `geom_histogram()`: familiar, but faceting uses more space. `geom_freqpoly()`: good for overlaying, hard to read with many groups. `geom_density()`: smooth, but can hide features.

Two Categorical Variables

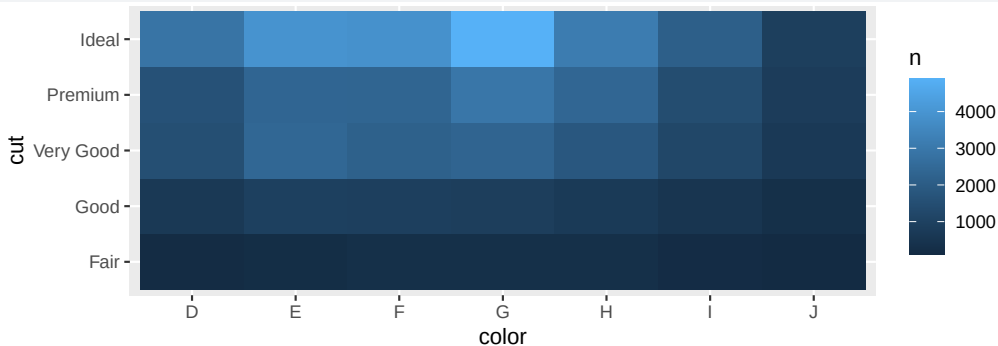
```
ggplot(diamonds, aes(x = cut, y = color)) +  
  geom_count()
```



Point size $\propto$ count. Alternative: `geom_tile()` with `fill = n`.

Heatmap with geom_tile()

```
diamonds |>  
  count(color, cut) |>  
  ggplot(aes(x = color, y = cut)) +  
  geom_tile(aes(fill = n))
```



Exercises (Two Categorical)

- 1 Rescale the count data to show distribution of `cut` within `color` (or vice versa).
- 2 Map `color` to `x` and `cut` to `fill` in a segmented bar chart. What insight?
- 3 Use `geom_tile()` with `dplyr` to show average departure delay by destination and month. What makes it hard to read?

Solutions (Two Categorical)

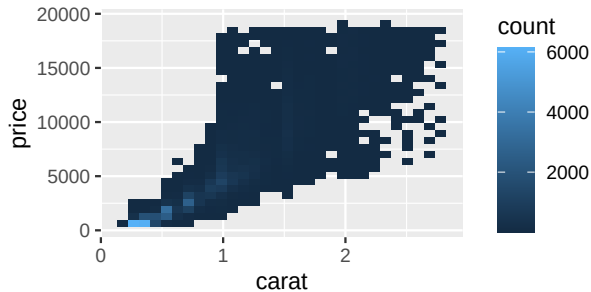
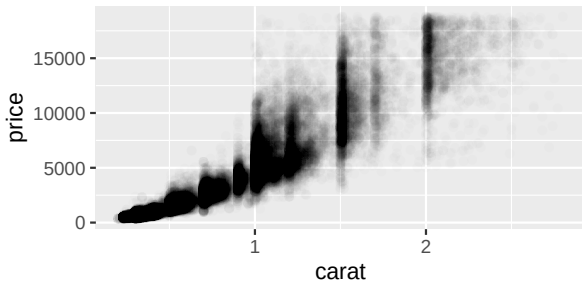
- 1 Compute proportions within groups:

```
diamonds |>
  count(color, cut) |>
  group_by(color) |>
  mutate(prop = n / sum(n)) |>
  ggplot(aes(x = color, y = cut, fill = prop)) +
  geom_tile()
```

- 2 Shows color composition per cut level. Compute with `count(cut, color)`.
- 3 Too many destinations — hard to read. Filter to busy destinations, or reorder by delay. Missing combos also create gaps.

Two Numerical Variables

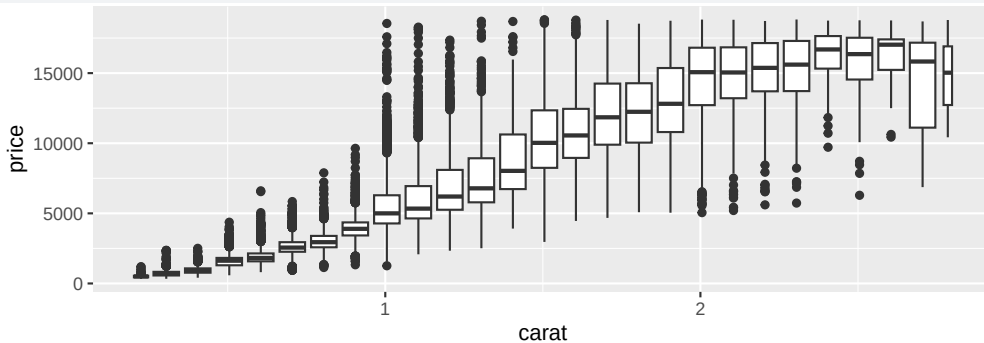
```
ggplot(smaller, aes(x = carat, y = price)) +  
  geom_point(alpha = 1/100)  
ggplot(smaller, aes(x = carat, y = price)) +  
  geom_bin2d()
```



Left: `alpha` reveals density. Right: `geom_bin2d()` bins into rectangles, colors by count.

Binned Boxplots

```
ggplot(smaller, aes(x = carat, y = price)) +  
  geom_boxplot(aes(group = cut_width(carat, 0.1)))
```



`cut_width(carat, 0.1)` bins carat into 0.1-wide groups. Shows how price varies across carat ranges.

Exercises (Two Numerical)

- 1 `cut_width()` vs. `cut_number()`: what to consider for frequency polygons?
- 2 Visualize distribution of `carat` partitioned by `price`.
- 3 How does price distribution of very large vs. small diamonds compare?
- 4 Combine techniques to visualize `cut`, `carat`, and `price` together.

Solutions (Two Numerical)

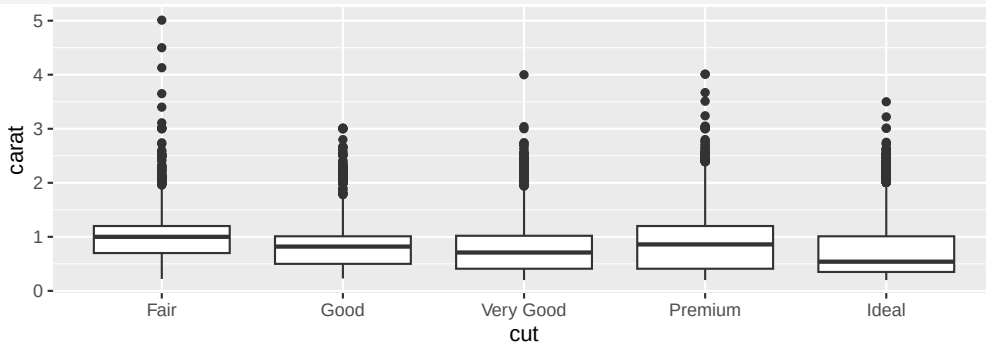
- ① `cut_width()` creates equal-width bins (some may have few points). `cut_number()` creates equal-count bins (widths vary). For frequency polygons, `cut_number()` ensures each line has enough data.
- ② `ggplot(diamonds, aes(x = price, y = carat)) + geom_boxplot(aes(group = cut_width(price, 2000)))`
- ③ Very large diamonds have high price variance. Fewer observations → less price competition → wider spread.
- ④ Example: `ggplot(diamonds, aes(x = carat, y = price, color = cut)) + geom_point(alpha = 0.1)` or facet by `cut`.

Patterns and Models

Confounding: Cut vs. Price

Why are Fair diamonds more expensive? **Carat confounds the relationship.**

```
ggplot(diamonds, aes(x = cut, y = carat)) +  
  geom_boxplot()
```

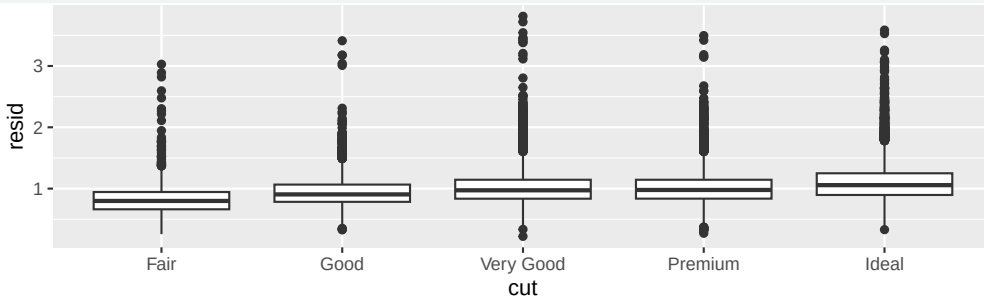


Fair diamonds tend to be larger → higher carat → higher price.

Removing Confounding with a Model

```
mod <- lm(log(price) ~ log(carat), data = diamonds)
diamonds2 <- diamonds |>
  mutate(resid = exp(residuals(mod)))

ggplot(diamonds2, aes(x = cut, y = resid)) +
  geom_boxplot()
```



After removing carat effect: better cut → higher price (as expected).